

Statistical Modelling for Horse Racing

Made by: Alexander Klasen

June 25, 2026



Contents

1	Introduction	1
2	Probabilistic Framework	2
2.1	Mathematical Model	2
2.2	Objective of the model	2
2.3	Market efficiency	3
3	Data	3
3.1	Result cards	3
3.2	Feature engineering	5
4	Model	6
4.1	The prediction task	6
4.2	Empirical Risk Minimisation	6
4.3	The learning algorithm	7
5	Implementation	8
5.1	Data Pipeline	8
5.2	Feature Engineering	9
5.3	Model Configuration	9
5.4	Evaluation	10
5.5	Results	10
6	Betting	16
6.1	Kelly criterion	16
6.2	Validation betting	16
6.3	Testset performance	17
7	Conclusion	19

1 Introduction

This project is my first deeper step into applied machine learning. I wanted to work on a problem that is not only theoretical, but also messy, uncertain, and connected to real decisions. Horse racing is a good setting for this because every race has a clear outcome, but the result is influenced by many noisy factors. It also connects naturally to sports betting, where probability estimates are not just abstract numbers, but can be compared directly with the odds offered by the market. The main idea of the project is to estimate the winning probability of each horse before a race takes place. Since exactly one horse wins each race, the outcome can be modelled as a categorical random variable. The model therefore has to assign a probability to every runner in a race, while keeping the probabilities meaningful as a full race-level distribution.

A central part of the project is the comparison with the betting market. Public odds can be interpreted as market-implied probabilities, so they provide a strong benchmark. If the model simply copies the market it will not reveal any betting edge. The hope is that the model can learn from historical race data and sometimes identify small inconsistencies in the probabilities implied by the odds. This makes the project about more than predicting winners. It is also about calibration, and whether a machine learning model can find useful edge in a rather efficient market. The final goal is not to claim a guaranteed profitable betting system, but to understand how such a system could be built, tested, and judged honestly on out-of-sample data.

2 Probabilistic Framework

2.1 Mathematical Model

Let

$$\Omega = \{r_t, r_{t-1}, \dots\}, \quad \text{where } r_t \text{ denotes the race at time } t.$$

For each race r_t , let

$$\mathcal{H}_{r_t} = \{h_1, \dots, h_{N_t}\},$$

where h_i denotes the i -th horse in race r_t and $N_t := |\mathcal{H}_{r_t}|$.

We consider a probability space

$$(\Omega, \mathcal{F}, P)$$

together with the filtration

$$\mathcal{F}_{t-1} = \sigma(r_{t-1}, r_{t-2}, \dots),$$

which represents all information available prior to race r_t .

For every horse $h_i \in \mathcal{H}_{r_t}$ we are given a feature vector

$$X_{r_t, h_i} \in \mathbb{R}^d, \quad \text{which is } \mathcal{F}_{t-1}\text{-measurable.}$$

Define the outcome random variables

$$Y_{r_t, h_i} = \mathbb{1}_{\{h_i \text{ wins race } r_t\}}.$$

Due to the structure of horse racing we have the constraint

$$\sum_{i=1}^{N_t} Y_{r_t, h_i} = 1 \quad \text{a.s.}$$

Strictly speaking, dead heats may occur, in which two horses share first place. However, these cases are sufficiently rare that we therefore ignore them in the analysis.

Our goal is to estimate the conditional win probabilities

$$p_{t,i} = P(Y_{r_t, h_i} = 1 \mid \mathcal{F}_{t-1}) \in (0, 1), \quad i = 1, \dots, N_t,$$

such that

$$\sum_{i=1}^{N_t} p_{t,i} = \sum_{i=1}^{N_t} P(Y_{r_t, h_i} = 1 \mid \mathcal{F}_{t-1}) = 1.$$

2.2 Objective of the model

For every race, each horse has its own winning odds. In Hong Kong horse racing, this means there exist odds

$$O_{r_t, h_i} \in \mathbb{Q}_{>1},$$

such that for every 1€ wagered you may win $O_{r_t, h_i}\text{€}$.

Given our statistical model, we will (theoretically) only bet if the expected profit is positive. Let

$$\mathcal{P}_{r_t, h_i} = \begin{cases} O_{r_t, h_i} - 1, & \text{if horse } h_i \text{ wins race } r_t, \\ -1, & \text{otherwise.} \end{cases}$$

be the net profit. We then require

$$E[\mathcal{P}_{r_t, h_i}] = p_{t,i} \cdot (O_{r_t, h_i} - 1) + (1 - p_{t,i}) \cdot (-1) = p_{t,i} \cdot O_{r_t, h_i} - 1 > 0.$$

In practice, the model does not condition directly on the full information set $\mathcal{F}_{t-1}$, but on a finite-dimensional summary in the form of feature vectors X_{r_t, h_i} (and, if needed, race-level features shared by all horses in a race). Equivalently, we approximate conditioning on $\mathcal{F}_{t-1}$ by conditioning on the smaller σ -algebra generated by the features available before race r_t , i.e. $\mathcal{A}_{t-1} = \sigma(X_{r_{t-1}, h_i}, X_{r_{t-2}, h_i}, \dots) \subseteq \mathcal{F}_{t-1}$. The learning task is therefore to construct an estimator

$$\hat{p}_{t,i}(\mathcal{A}_{t-1}) \approx P(Y_{r_t, h_i} = 1 \mid \mathcal{F}_{t-1})$$

that yields well-calibrated probabilities and, by extension, supports the previously established positive expected-profit criterion. This corresponds to assuming that market-implied probabilities derived from public odds may be mispriced and that the model can learn corrections and exploit such deviations.

2.3 Market efficiency

The HKJC uses a parimutuel betting system. This means that all bets are pooled, and the operator pays out a specified percentage of the pool to the winners as dividends. In the specific case of betting on the winning horse, the HKJC applies a house take of 17.5%. Therefore, the final payout, and thus the winning odds, depends entirely on the amount of money in the pool after deduction of the house take. Beyond that, it relies on the hypothesis that public bettors may exhibit systematic biases, such as recency bias, or bandwagon effects, thereby creating exploitable deviations from true win probabilities.

3 Data

3.1 Result cards

The Hong Kong Jockey Club publishes race results in a structured tabular format on its website.

Racing Information (Local) - Results

Sha Tin: 1 2 3 4 5 6 7 8 9 10 All Races

Race Meeting: 22/03/2026 Sha Tin

22/03/2026 go

Sectional Time & Position

RACE 1 (535)

Class 4 - 1600M - (60-40)
LUGER HANDICAP
HK\$ 1,170,000

Going : GOOD
Course : TURF - "A" Course
Time : (24.67) (47.53) (1:11.45) (1:34.71)
Sectional Time : 24.67 22.86 23.92 23.26
12.09 11.83 11.36 11.90

Multi Angle Race Replay Pass Through Analysis

Pla.	Horse No.	Horse	Jockey	Trainer	Act. Wt.	Declar. Horse Wt.	Dr.	LBW	Running Position	Finish Time	Win Odds
1	8	MEGA MASTERMIND (L044)	B Avdulla	J Richards	125	1230	11	-	1 1 1 1	1:34.71	7.3
2	10	CIRCUIT MARSHAL (K526)	C L Chau	M Newnham	122	1147	5	1/2	12 12 11 2	1:34.80	22
	3	FLYING BOOM (K327)	M F Poon	C W Chang	128	1095	1	1	6 7 6 3	1:34.87	8.1
4	9	MULTISUPERSTAR (L163)	A Badel	A S Cruz	125	1110	13	1-1/4	11 10 8 4	1:34.91	132
5	1	ROBOT STAR (K559)	Z Purton	K L Man	135	1062	7	1-3/4	7 6 5 5	1:35.00	2.8
6	3	FORZA TORO (K355)	H Bowman	C Fownes	134	1122	9	2-1/2	8 3 3 6	1:35.12	3.6
7	12	FIGHTING MACHINE (J015)	K Teetan	P C Ng	122	1084	4	2-3/4	13 13 12 7	1:35.14	33
8	4	THE ABSOLUTE (H438)	L Ferraris	K H Ting	133	1153	2	3-1/4	9 9 9 8	1:35.21	25
9	13	MATSU VICTOR (H472)	C Y Ho	K W Lui	121	1086	10	3-1/4	5 5 7 9	1:35.24	22
10	14	WINNING DIAMOND (K216)	M L Yeung	C H Yip	116	1024	8	4	2 4 4 10	1:35.33	28
11	2	BUCEPHALAS (K096)	K C Leung	W K Mo	134	1167	12	5	4 2 2 11	1:35.52	20
12	5	GLORIOUS SUCCESS (K062)	D B McMonagle	D Eustace	132	1075	14	6	14 14 14 12	1:35.67	19
13	7	GRAND TURBO (L138)	R Kingscote	C S Shum	128	1065	3	9-3/4	10 11 13 13	1:36.26	160
14	11	LUCKY SECRET (L073)	E C W Wong	D A Hayes	121	1239	6	12-1/4	3 8 10 14	1:36.68	135

Example result card. Source: racing.hkjc.com

In the following, I explain the key information with predictive power that can be extracted from these result cards.

1. The **date** on which the race took place, which is used to ensure strict temporal ordering of training and test data.
2. The **venue** of the race, either Sha Tin or Happy Valley.
3. The **class** of the race, which describes the performance level of the participants. The classes are ranked in descending order:

Class 5 $\longrightarrow$ Class 4 $\longrightarrow$ Class 3 $\longrightarrow$ Class 2 $\longrightarrow$ Class 1.

4. The **distance** of the race.
5. The **course type**, distinguishing between turf and all-weather tracks (sand).
6. The **going**, which indicates the condition of the racing surface. The terminology depends on the surface type.

For turf tracks:

Heavy $\longrightarrow$ Soft $\longrightarrow$ Yielding $\longrightarrow$ Good to Yielding $\longrightarrow$ Good $\longrightarrow$ Good to Firm $\longrightarrow$ Firm

For all-weather tracks:

Wet Slow $\longrightarrow$ Slow $\longrightarrow$ Good $\longrightarrow$ Sealed $\longrightarrow$ Wet Fast

7. The final **placing** of the horse.
8. The **horse** name/ID.
9. The **jockey** who rode the horse.
10. The **trainer** of the horse.
11. The **actual weight** carried by the horse.
12. The **declared weight** of the horse.
13. The **draw** from which the horse started.
14. The **lengths behind the winner (LBW)** measure the distance between a horse and the winner at the finish, expressed in horse lengths (approximately 2.4 metres). A smaller value indicates a closer finish.
15. The **running position** of the horse at each section of the race.
16. The **finish time** of the horse.

3.2 Feature engineering

Before starting this project, I had limited insight into horse racing, and HKJC in particular. Since result cards mostly provide hard facts, substantial feature engineering is required to extract predictive indicators. The race-level hard facts extracted in the previous subsection, such as venue, class, distance, course type, going, and field size, are used directly as features and are not repeated below. Where applicable, rolling window sizes are treated as hyperparameters denoted $P_i \in \mathbb{N}$, where $i \in \mathbb{N}$.

1. The draw from which the horse starts, normalized by field size so that draws are comparable across races with different numbers of participants.
2. The horse's win rate over up to the last P_1 races, as an indicator of current form.
3. The horse's average finishing position over up to the last P_2 races. This carries finer-grained information than the win rate (e.g., consistent runners-up are distinguishable from consistent back-markers, both of whom may have a win rate of zero).
4. The jockey's win rate over up to the last P_3 races, indicating how well the horse is ridden.
5. The trainer's win rate over up to the last P_4 races, indicating the effectiveness of the training approach.
6. The declared weight of the horse, which reflects bodyweight and indicates condition and fitness.
7. The actual weight carried by the horse, which reflects the handicap assignment and indicates the horse's official rating.
8. Days since the last race, indicating fitness and freshness, and potentially past injuries if the gap is unusually long.
9. Average performance under specific race conditions, computed separately for each of the following:
 - distance,
 - going,
 - course type.

Each is the horse's average position conditioned on the respective variable. Sparsity is expected for horses with few prior races, and missing values are handled as `NaN`.

10. Average lengths behind the winner (LBW) over up to the last P_5 races, capturing strong performances even in close losses.
11. Historical average quantile of running position at each section, averaged over the horse's past races. This characterises the horse's typical running style (front-runner, mid-pack, closer), which interacts with race distance and pace. Races have different section counts depending on distance, so sections absent from a past race are treated as `NaN`.
12. Average class level in which the horse has competed, since class changes between races may indicate underdog or favourite status due to skill gaps.
13. Raw odds converted to implied win probabilities, corrected for the house take. These aggregate public belief about the likely winner and may contain signals not present in the result cards, such as insider information.

4 Model

4.1 The prediction task

As established in the probabilistic framework (Chapter 2), the goal of the model is to estimate the conditional win probabilities $p_{t,i} = P(Y_{r_t, h_i} = 1 \mid \mathcal{F}_{t-1})$ for each horse $h_i \in \mathcal{H}_{r_t}$.

Since exactly one horse wins each race, the observed outcome is described by the labeling function

$$f : \mathcal{H}_{r_t} \rightarrow \{0, 1\}, \quad h \mapsto \mathbb{1}_{\{h^* \text{ wins race } r_t\}},$$

which is the probability mass function of the Dirac measure δ_{h^*} on $\mathcal{H}_{r_t}$, where h^* denotes the winning horse. A model with perfect foresight would recover f exactly. Since the race outcome is unknown prior to the race, the best we can do is estimate the conditional win probabilities $p_{t,i}$, so the model instead learns $\hat{f}$, an estimated probability mass function over Ω , satisfying the constraint

$$\sum_{i=1}^{|\mathcal{H}_{r_t}|} \hat{f}(X_{r_t, h_i}) = 1,$$

such that $\hat{f}(X_{r_t, h_i}) \approx p_{t,i}$ for all $h_i \in \mathcal{H}_{r_t}$, which in the limit of perfect prediction collapses to f .

4.2 Empirical Risk Minimisation

Since this project boils down to a supervised learning task, the machine learning model learns $\hat{f}$ by following the principles of empirical risk minimisation. This means that the learning algorithm chooses a function from a hypothesis class $\mathcal{C}$ that minimises the empirical risk over the training set $\mathcal{T}$, where $\mathcal{T}$ consists of tuples containing feature vectors and the corresponding label for a specific race r_t . Thus,

$$\hat{f} = \arg \min_{w \in \mathcal{C}} R_{emp}(w),$$

where

$$R_{emp}(w) = \frac{1}{n} \sum_{i=1}^n L(w(x_i), y_i), \quad (x_i, y_i) \in \mathcal{T}$$

describes the average loss of the hypothesis h and, by the law of large numbers, approximates its expected loss. Since we are dealing with a categorical classification problem and want the model to output probabilities, it is natural to use the categorical cross-entropy loss, defined as

$$L(p, y) = -y \log(p), \quad y \in \{0, 1\}, p \in [0, 1].$$

In our case, this works well because we want to penalise predictions close to 0 or 1 that turn out to be wrong, which encourages well-calibrated probabilities. The values output by the model do not necessarily have the desired form, since they may be negative, exceed 1, or fail to define a probability mass function. We therefore apply the softmax function to transform these values into the desired probabilities, which for a specific race are defined as

$$\hat{p}_i = \frac{e^{\hat{f}(x_i)}}{\sum_{j=1}^{|\mathcal{H}_{r_t}|} e^{\hat{f}(x_j)}}.$$

Since exactly one horse wins per race, only the winning horse contributes to the loss, as all other terms vanish when $y_i = 0$. The softmax constraint ensures that increasing the predicted probability

of the winner necessarily suppresses the probabilities of all other horses in the race, implicitly penalising overconfident predictions for losing horses. The loss function we use therefore takes the form

$$L(x_i, y_i) = -y_i \cdot \left(\hat{f}(x_i) - \log \left(\sum_{j=1}^{|\mathcal{H}_{r_t}|} e^{\hat{f}(x_j)} \right) \right), \quad (x_i, y_i) \in \mathcal{T}.$$

4.3 The learning algorithm

Since the chosen features and labels naturally result in tabular data, gradient-boosted decision trees are a natural choice for predicting future race odds. In this setting, the hypothesis class $\mathcal{C}$ consists of decision trees. A decision tree makes predictions by recursively splitting the feature space into regions with similar outcomes. However, a single tree is usually neither expressive nor stable enough to capture all relevant patterns reliably. We therefore use boosting, which combines many weak learners into a stronger learner. To formalise this idea, we introduce the notion of weak learnability.

Definition 4.1 (γ -Weak Learnability). A learning algorithm l is called a γ -weak learner for a class $\mathcal{C}$ if there exists a function $m_{\mathcal{C}} : (0, 1) \rightarrow \mathbb{N}$ such that, for every $\delta \in (0, 1)$, every distribution d over $\mathcal{X}$, and every labelling function f , the following holds: if the realizability assumption is satisfied with respect to $\mathcal{C}$, d , and f , then, when the learning algorithm is run on a training set $\mathcal{T}$ of size $m \geq m_{\mathcal{C}}(\delta)$ consisting of i.i.d. examples generated by d and labelled by f , it returns a hypothesis w such that

$$P\left(E[L(w(x), f(x))] \leq \frac{1}{2} - \gamma\right) \geq 1 - \delta,$$

where the expectation is taken with respect to $x \sim d$.

In our case, the weak learners are decision trees, which, as mentioned above, are less expressive on their own. Here, we do not need to focus on the specific value of γ for each individual tree. We only require each tree to perform better than random guessing, meaning that $\gamma > 0$, although its exact value may vary across trees. Therefore, using boosting, we seek the following general form for our conditional win-probability predictor:

$$\hat{f}(X_{r_t, h_i}) = \sum_{k=1}^M c_k w_k(X_{r_t, h_i}), \quad w_k \in \mathcal{C}, \quad c_k \in \mathbb{R}, \quad k = 1, \dots, M,$$

where M represents a finite natural number that is chosen optimally during training. This representation is generally constructed iteratively as follows:

$$\begin{aligned} \hat{f}_0 &= \arg \min_{w \in \mathcal{C}} \sum_{i=1}^{|\mathcal{T}|} L(w(x_i), y_i), \\ \hat{f}_m &= \hat{f}_{m-1} + \arg \min_{w \in \mathcal{C}} \sum_{i=1}^{|\mathcal{T}|} L(\hat{f}_{m-1}(x_i) + w(x_i), y_i). \end{aligned}$$

Depending on the hypothesis class, it is clearly infeasible to always find the exact minimiser. We therefore apply gradient descent as an approximation to the initial goal of the iteration. This means that, instead, we look for constants c_m , $m \in \{1, \dots, M\}$, such that

$$\hat{f}_m = \hat{f}_{m-1} - c_m \sum_{i=1}^{|\mathcal{T}|} \frac{\partial L(\hat{f}_{m-1}(x_i), y_i)}{\partial \hat{f}_{m-1}(x_i)}, \quad c_m \in \mathbb{R}^+.$$

In the m -th iteration step, we would then set

$$c_m = \arg \min_{r \in \mathbb{R}^+} \sum_{i=1}^{|\mathcal{T}|} L \left(\hat{f}_{m-1}(x_i) - r \frac{\partial L(\hat{f}_{m-1}(x_i), y_i)}{\partial \hat{f}_{m-1}(x_i)}, y_i \right).$$

To return to the boosting representation, a decision tree w_m is fitted to the negative gradients $-\frac{\partial L(\hat{f}_{m-1}(x_i), y_i)}{\partial \hat{f}_{m-1}(x_i)}$, treated as pseudo-residuals, and the constant c_m is then solved via the minimisation above with the tree substituted in place of the gradient direction; fitting a tree to the pseudo-residuals rather than using the gradients directly introduces a further level of approximation, since the tree can only imperfectly represent the gradient vector.

In the special case of decision trees, however, J. H. Friedman proposed not only looking for a single constant r that scales the whole tree, but instead to optimise a separate constant per leaf. Since every learner attains the additive representation

$$w_k(X_{r_t, h_i}) = \sum_{j=1}^{J_k} b_{kj} \mathbb{1}_{\{X_{r_t, h_i} \in R_{kj}\}}, \quad b_{kj} \in \mathbb{R}, \quad j = 1, \dots, J_k, \quad J_k \in \mathbb{N},$$

where R_{kj} denotes the j -th leaf region of the k -th tree and J_k is the leaf count, the update becomes

$$\hat{f}_m = \hat{f}_{m-1} + \sum_{j=1}^{J_m} b_{jm} \mathbb{1}_{\{\cdot \in R_{jm}\}}.$$

Since the leaf regions R_{jm} are disjoint, each training point x_i falls into exactly one region, so the loss decomposes into J_m independent subproblems and the optimal leaf values can be found separately by solving

$$b_{jm} = \arg \min_{b \in \mathbb{R}} \sum_{x_i \in R_{jm}} L(\hat{f}_{m-1}(x_i) + b, y_i).$$

5 Implementation

This chapter describes how the modelling framework developed above is realised as an executable pipeline. Rather than documenting the code line by line, it focuses on the design decisions that are not self-evident from the source and connects each of them back to the statistical assumptions of the project. The pipeline proceeds in four stages: acquiring and cleaning the data, constructing features under a strict temporal discipline, configuring the learning algorithm with a race-aware objective, and evaluating the model against the market it aims to outperform.

5.1 Data Pipeline

Race results were collected directly from the HKJC website, with each race card stored as a raw HTML file before any parsing. Separating acquisition from parsing serves a practical purpose: the scraped pages form an immutable local record, so the parsing logic can be revised and re-run without repeatedly querying the HKJC servers. The parsed data is organised into two tables, one row per race and one row per horse per race, linked by a race identifier of the form YYYY-MM-DD_RaceNo. The date-first format with a zero-padded race number is deliberate: it makes lexicographic string ordering coincide with chronological ordering, which the entire feature pipeline later relies on. Qualitative quantities recorded on the result cards, such as winning margins and class labels, are mapped to numeric values at this stage so that all downstream computation operates on a consistent quantitative representation.

5.2 Feature Engineering

The defining constraint of the feature construction is temporal: for any race r_t , every historical statistic is computed using only races strictly preceding r_t . This is what makes the estimator a function of the pre-race information set $\mathcal{A}_{t-1}$ rather than of the outcome it is meant to predict, and it is enforced uniformly by restricting each computation to rows whose identifier precedes the current race. Condition-specific features, such as a horse’s average finishing position over races of the same distance, going, or course, follow the same rule, intersecting the horse’s permissible history with the set of races matching the relevant condition.

A subtlety arises with the sectional running positions, which characterise a horse’s typical running style. Because races of different lengths are divided into different numbers of sections, raw positions are not comparable across races. Each position is therefore normalised by the field size of the race in which it was recorded, and sections absent from a past race are excluded rather than imputed, so that a horse’s running-style profile is not distorted by races in which a given section does not exist. The rolling window length over which each historical statistic is averaged is treated as a tunable hyperparameter and selected by out-of-sample performance. Two regimes emerged. For form indicators such as win rate and average class, a single most recent observation performed as well as any longer window, indicating that these quantities carry little additional signal once the latest race is known. For average finishing position and its condition-specific variants, a window of five races was optimal, with shorter windows too noisy and longer windows diluting recent form.

Finally, the public odds are incorporated as a feature by converting them to implied win probabilities,

$$\hat{p}_i = (1 - r) \cdot \frac{1}{O_i},$$

where O_i is the decimal win odds of horse h_i and $r = 0.175$ is the HKJC house take. As established in Section 2.3, the uniform takeout of a parimutuel market makes this scaling sufficient to recover a correctly normalised probability vector, so no further normalisation across the field is required.

5.3 Model Configuration

The model is a gradient-boosted decision tree ensemble. Because the loss derived in Section 4.2 is defined over the horses of a race jointly rather than over independent observations, the standard objectives provided by the library cannot be used. The gradients and Hessians are instead supplied by a custom objective that treats each race as a single softmax group. Given the current raw scores, the per-race softmax yields predicted probabilities $\hat{p}_i$. For a fixed race r_t with field size N_t , the gradient and Hessian of the categorical cross-entropy with respect to the raw score of horse h_i are

$$G_{t,i} = -y_{t,i} + \hat{p}_{t,i} \sum_{j=1}^{N_t} y_{t,j}, \quad H_{t,i} = \hat{p}_{t,i} (1 - \hat{p}_{t,i}) \sum_{j=1}^{N_t} y_{t,j},$$

where the sum runs over all horses in the race. The sum is left in its general form rather than reduced to one: although the probabilistic model of Section 2.1 idealises each race as having a single winner, the loss imposes no such restriction, and keeping the general form allows the rare dead heats present in the data to contribute both winning horses to the gradient rather than forcing an arbitrary tie-break.

This race-grouped objective constrains the rest of the configuration in one important way: row subsampling must be disabled. A softmax group is only meaningful if all horses of a race are present together, so sampling individual rows would fragment the groups and corrupt the gradient. The remaining hyperparameters reflect the high noise inherent in race outcomes and are chosen

to favour shallow, strongly regularised trees: a small number of leaves, a substantial minimum of observations per leaf, an L_2 penalty on the leaf values, and a modest learning rate combined with column subsampling. The string-valued attributes (venue, going, course, jockey, and trainer) are passed to the model as native categorical features, allowing splits to exploit their high cardinality directly.

5.4 Evaluation

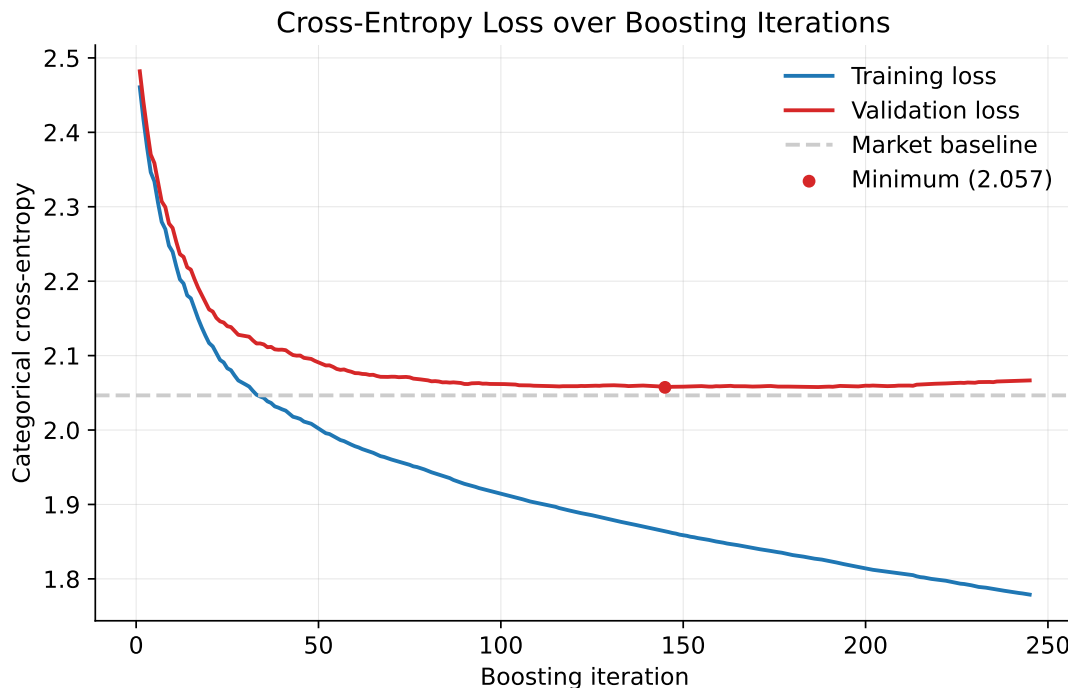
Model selection is driven by the per-race categorical cross-entropy, which coincides with the training loss and rewards probability mass placed on the horse that actually won. Alongside it, a top-1 accuracy is recorded as a statistic: the fraction of races whose winner was assigned the highest predicted probability. The two are deliberately distinct, since a model may rank favourites well while remaining poorly calibrated, or the reverse; only cross-entropy is used for early stopping, with top-1 accuracy serving to interpret behaviour rather than to optimise it.

The evaluation is anchored throughout to the market. Both metrics are computed not only for the model but also for the implied probabilities derived from the public odds, giving the loss and ranking accuracy the market itself achieves on the same validation races. The model is thus measured against a concrete and demanding baseline rather than an abstract one.

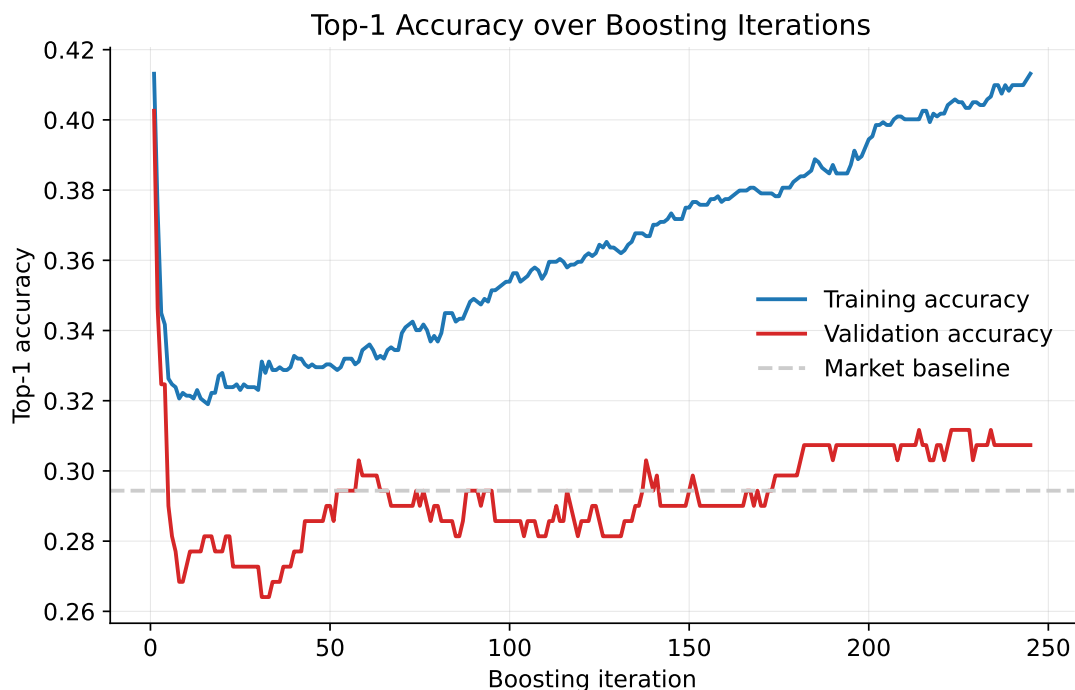
To preserve the temporal discipline that governs the features, the train-validation split is chronological and performed at the level of whole races: the earliest 80% of races are used for training and the remainder for validation and testing, so that every validation race occurs strictly after every training race and no race is divided between the two sets. Training proceeds until the validation cross-entropy fails to improve for a fixed number of rounds, and the best configuration is saved.

5.5 Results

The data set consists of 1539 races and is split into 80/15/5 training, validation, and test sets. With the model configuration described above, the validation loss evolves as follows.

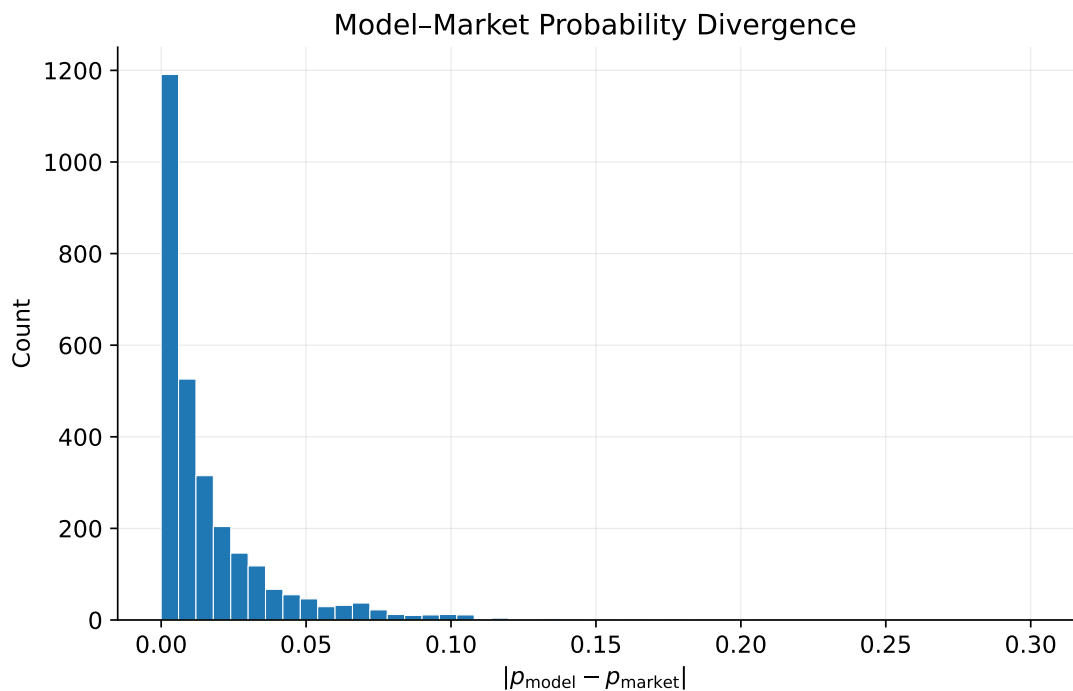


The validation loss at the best boosting round is close to the loss achieved by the implied-odds baseline. This suggests that the model does not dramatically outperform the market in terms of cross-entropy. At the same time, the model appears to distribute probability mass somewhat more conservatively instead of assigning very high probability to a single likely winner. This is not necessarily undesirable, because the aim is not only to identify the most likely winner, but also to detect horses whose market odds may underestimate their true winning probability. Since such horses still need to win for a betting strategy to be profitable, it is also useful to examine how often the model assigns the highest probability to the actual winner.



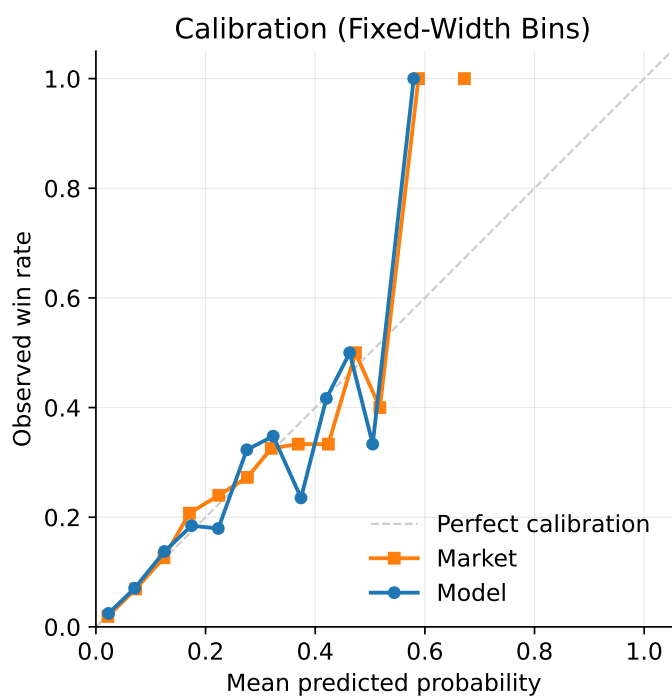
Compared with the winner-selection rate implied by the market probabilities, the model performs around the same, with 29%. However, this result alone is not sufficient to justify a betting strategy, since profitability depends not only on selecting winners but also on the prices at which those winners are backed.

For a profitable betting strategy, the model must also be able to diverge from the market-implied probabilities: simply reproducing the market would identify the same prices as fair and therefore leave no exploitable edge.



Before considering statistics that are more directly related to betting edge, it is important to check whether the predicted probabilities are calibrated. Since implied market probabilities are among the most influential features in the training process, any systematic miscalibration in the market could also be transferred to the model.

To assess calibration, all predicted probabilities are first grouped into half-decile bins. For each bin, the predicted probability range is compared with the empirical fraction of horses that actually won.

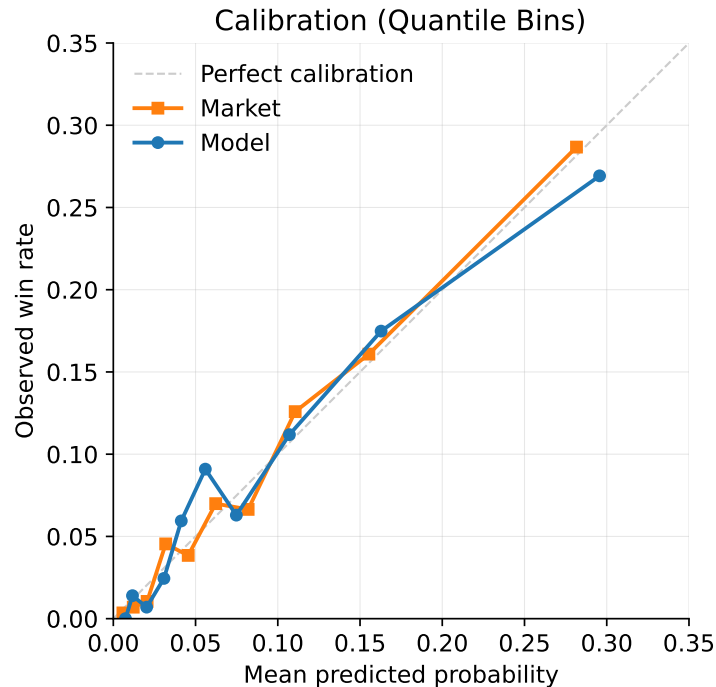


In the lower probability bins, both the model predictions and the implied market probabilities lie close to the ideal calibration line. At higher probabilities, the curves become less stable or disappear entirely, mainly because of data sparsity: both the model and the market rarely assign very high winning probabilities. The table below reports the number of observations and winners in each bin, illustrating this sparsity.

Table 1: Model calibration by fixed-width probability bin (validation set).

Probability range	Winners	Total	Win rate	Bin mean
0.00–0.05	36	1467	0.025	0.023
0.05–0.10	45	639	0.070	0.071
0.10–0.15	40	291	0.137	0.125
0.15–0.20	31	168	0.185	0.174
0.20–0.25	21	117	0.179	0.223
0.25–0.30	21	65	0.323	0.275
0.30–0.35	16	46	0.348	0.323
0.35–0.40	8	34	0.235	0.374
0.40–0.45	10	24	0.417	0.420
0.45–0.50	3	6	0.500	0.463
0.50–0.55	1	3	0.333	0.505
0.55–0.60	1	1	1.000	0.580
0.60–1.00	0	0	–	–

This visualisation issue can be mitigated by using fixed bin counts, and therefore variable bin sizes, instead of fixed probability intervals. These bins are slightly less direct to interpret, but they ensure that each bin contains a comparable number of observations.



Under this quantile-based view, the model's observed win frequencies again remain close to the

Table 2: Model calibration by quantile bin (validation set, 10 bins).

Probability range	Winners	Total	Win rate	Bin mean
0.0040–0.0093	0	286	0.000	0.007
0.0093–0.0147	4	286	0.014	0.012
0.0147–0.0259	2	286	0.007	0.020
0.0259–0.0360	7	286	0.024	0.031
0.0360–0.0483	17	286	0.059	0.041
0.0483–0.0645	26	286	0.091	0.056
0.0645–0.0863	18	286	0.063	0.075
0.0863–0.1312	32	286	0.112	0.107
0.1312–0.2032	50	286	0.175	0.163
0.2032–0.5797	77	286	0.269	0.296

ideal calibration line. This suggests that the calibration error is small and comparable to that of the market, as shown by the expected calibration error (ECE):

	Model	Market
Fixed-width	0.81%	0.65%
Quantile	1.38%	0.87%

This corresponds to a difference of 0.16 percentage points under fixed-width binning and 0.51 percentage points under quantile binning. Therefore, the cross-entropy gap to the market appears to reflect sharpness and the difficulty of confidently recreating the Dirac measure for each race, as discussed in the prediction-task subsection, rather than model miscalibration.

This finding is somewhat surprising, since at first glance it weakens the initial expectation that public bettors exhibit a strong favourite–longshot bias. The market calibration at higher probabilities is not far from the ideal line. However, the small number of observations in this region means that the apparent absence of bias cannot be taken as conclusive.

Since a betting strategy rule has already been established, it is natural to ask how often the positive expected-profit rule introduced in Chapter 2 is triggered on the validation set, given the model’s predicted probabilities.

Share of horses with positive expected profit: 23.9%

This is also surprising, given the size of the house take. Filtering the races by this criterion gives a different picture of calibration for the selected subset.

The model’s data points consistently lie below the optimal calibration line. This means that, within this subset, the model assigns probabilities that are too high, which could be problematic for later bet sizing. Therefore, it makes sense to try to improve its accuracy on this specific subset.

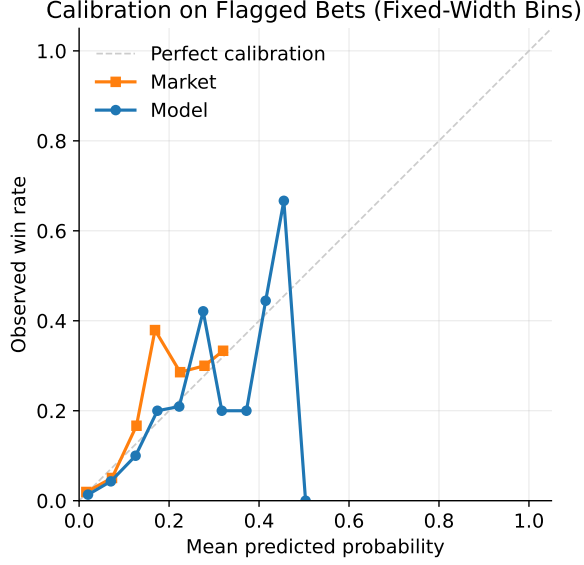


Figure 1: Model calibration on flagged bets (fixed-width bins).

Range	Win	Tot	Rate	Mean
0.00–0.05	5	370	0.014	0.020
0.05–0.10	4	93	0.043	0.071
0.10–0.15	6	60	0.100	0.126
0.15–0.20	11	55	0.200	0.174
0.20–0.25	9	43	0.209	0.223
0.25–0.30	8	19	0.421	0.276
0.30–0.35	3	15	0.200	0.317
0.35–0.40	3	15	0.200	0.372
0.40–0.45	4	9	0.444	0.415
0.45–0.50	2	3	0.667	0.455
0.50–0.55	0	1	0.000	0.503
0.55–1.00	0	0	–	–

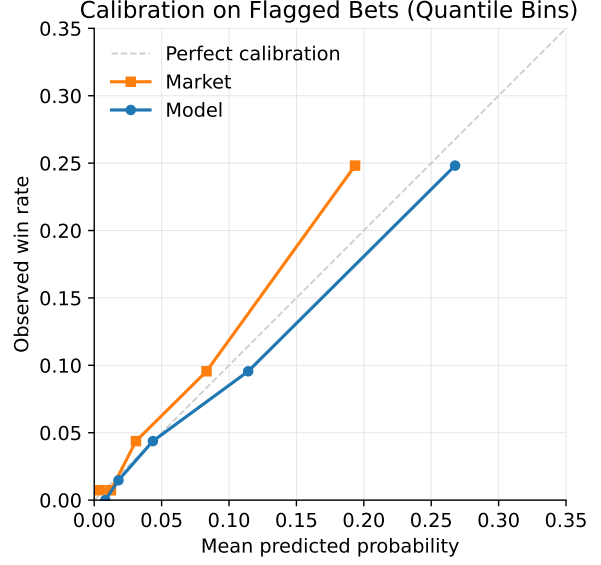


Figure 2: Model calibration on flagged bets (quantile bins).

Range	Win	Tot	Rate	Mean
0.0000–0.0106	0	136	0.000	0.008
0.0106–0.0293	2	136	0.015	0.018
0.0293–0.0650	6	137	0.044	0.044
0.0650–0.1683	13	136	0.096	0.114
0.1683–0.5034	34	137	0.248	0.268

Unfortunately, all calibration attempts failed, because manipulating individual probabilities is restrictive in this setting. Each race must still satisfy the condition that there is always a winner, so the probabilities within a race must sum to one. A standard calibration method such as Platt scaling therefore failed: after calibration, the probabilities had to be renormalised, which distorted the intended adjustment and generally worsened the calibration level.

The next idea was to restrict the model output by combining it with the market-implied probabilities. This can be interpreted as trusting the model only up to a factor $\alpha \in [0, 1]$, leading to the convex combination

$$p_{\text{new}} = (1 - \alpha) \cdot p_{\text{model}} + \alpha \cdot p_{\text{market}}.$$

This method is well suited to the setting because it keeps the probability distribution within each race valid. It also actively reduces the number of bets flagged as potentially mispriced, since trusting the market probabilities fully corresponds to seeing no edge. Choosing the parameter α based on total cross-entropy or on the win rate of the flagged bets is not particularly meaningful, since this adjustment does not directly calibrate the probabilities themselves. Instead, it controls the degree to which the model is allowed to diverge from the market. Its success should therefore be evaluated

through the ROI generated by the remaining, reduced set of model-implied edges. A minimum probability threshold of $p \geq 0.1$ is additionally imposed. Although the positive expected-profit rule may flag low-probability horses, such bets rely on rare large payouts to realise their edge. At the present data set size the number of flagged longshots is too small for their realised win frequency to converge to the underlying probability, so a handful of outcomes would dominate the measured return. Excluding them removes a source of variance that the sample is too small to average out, rather than removing genuinely unprofitable bets. This evaluation is discussed in the next section.

6 Betting

6.1 Kelly criterion

In HKJC win betting, as in many other forms of sports betting, the return of a single bet is binary: either the selected horse wins, or the stake is lost. Therefore, choosing which horse to bet on is only one part of the problem. Bet sizing is equally important, since it determines how quickly a strategy can either lose its bankroll or compound profitable edges. The Kelly criterion provides a principled way to allocate risk and maximise the expected growth rate of wealth, provided that the probabilities used are well calibrated. For decimal odds, the fraction of the bankroll to wager is given by

$$f = p - \frac{1-p}{O-1}, \quad p \in [0, 1], \quad O \in \mathbb{Q}_{>1}.$$

Here, p denotes the predicted probability of winning and O denotes the corresponding decimal odds. Since Kelly bets are highly sensitive to errors in the estimated probabilities, practitioners often use a fractional Kelly criterion. This simply scales the Kelly stake by an additional factor in $(0, 1)$, reducing risk at the cost of slower expected bankroll growth.

6.2 Validation betting

When choosing the parameter α for the convex combination, the first objective is to find a range in which the total ROI on the validation set is both stable and as high as possible. Isolated spikes in ROI may not transfer to unseen data, so stability across neighbouring values of α is more important than a single maximum. The following table reports the total ROI obtained by betting on all flagged horses in temporal order, with the bankroll updated after each bet.

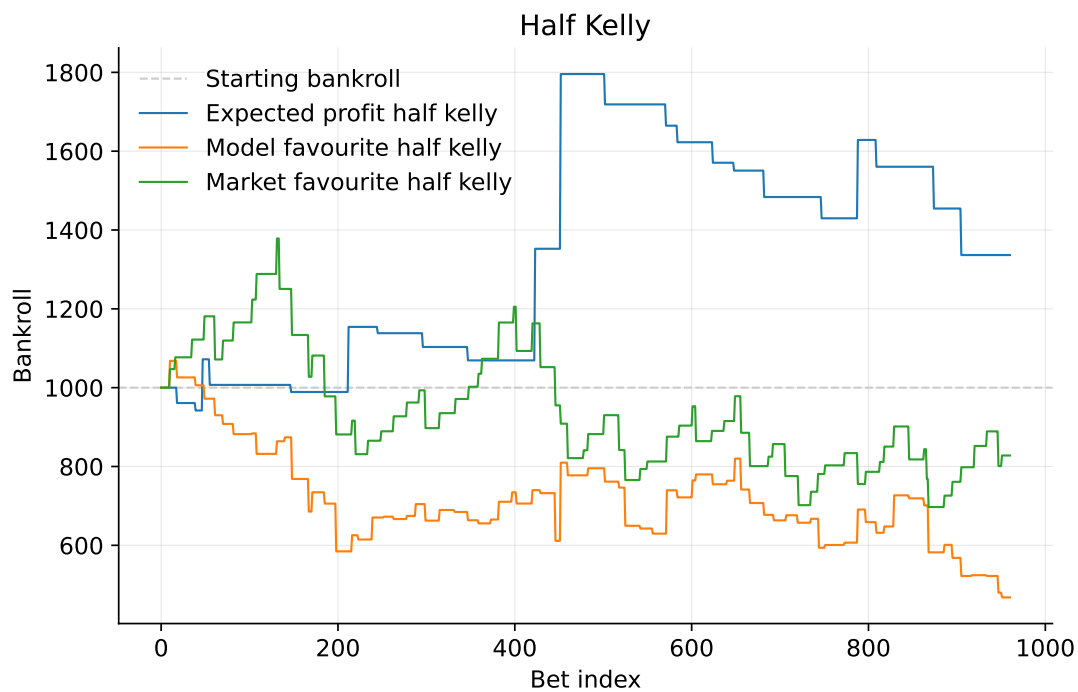
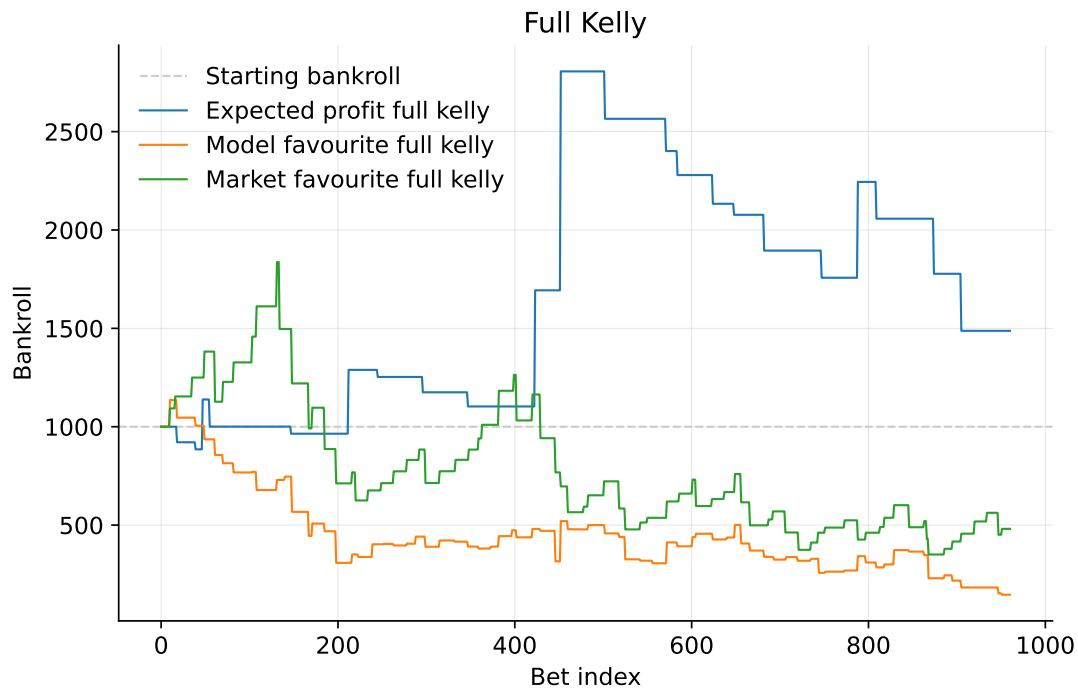
Based on this observation, I continue with $\alpha = 0.55$. This value sits near the peak of validation performance while still retaining 55 bets, which is a large enough sample for the result not to be dominated by the variance of a handful of outcomes. Larger values of α yield higher final bankrolls, but only on rapidly shrinking bet counts, so their apparent advantage cannot be distinguished from noise. This already highlights an important limitation of the project: the data set is relatively small. The training set must be large enough for the model to learn the general win-loss structure, while the validation set must contain enough flagged bets to choose α without relying on a noisy estimate of ROI. At the same time, the test set should not be too far removed from the training period, since structural changes in the racing environment or newly emerging patterns may not be captured by the model. These competing requirements motivate the chosen train-validation-test split. Consequently, the test-set results should not be interpreted as definitive, but rather as indicative results that may change as the data set grows.

Table 3: Validation-set betting performance across shrinkage parameter α . Final bankroll is measured from an initial bankroll of 1000.

α	Bets	Win rate	Full Kelly	Half Kelly
0.00	220	0.209	1617.2	1555.8
0.05	207	0.203	1286.3	1373.8
0.10	185	0.205	1315.9	1408.1
0.15	176	0.199	1325.6	1413.0
0.20	165	0.200	1354.0	1413.2
0.25	145	0.200	1328.4	1403.1
0.30	130	0.200	1359.8	1406.4
0.35	119	0.202	1549.3	1499.8
0.40	101	0.188	1425.7	1439.6
0.45	88	0.182	1330.1	1389.6
0.50	73	0.205	1759.3	1575.0
0.55	55	0.236	2396.2	1812.9
0.60	37	0.270	2693.4	1882.2
0.65	26	0.269	2073.2	1605.6
0.70	14	0.214	1256.0	1180.0
0.75	7	0.286	1514.0	1263.0
0.80	7	0.286	1514.0	1263.0
0.85	6	0.333	1785.0	1366.0
0.90	0	0.000	1000.0	1000.0
0.95	0	0.000	1000.0	1000.0
1.00	0	0.000	1000.0	1000.0

6.3 Testset performance

The following graph compares three different betting strategies. The first is the expected-profit strategy discussed above. The second simply backs the market favourite, while the third backs the horse assigned the highest winning probability by the model. The latter two strategies could be refined further, but they are introduced here only as benchmarks for the expected-profit strategy.



These results are computed on the held-out test set, which the model never saw during training and on which no parameters were tuned; the shrinkage and staking parameters were fixed beforehand on the validation set. The out-of-sample outperformance is therefore not an artefact of in-sample fitting.

7 Conclusion

At first sight, the results look promising. Under both staking schemes, the expected-profit strategy finishes above the model-favourite and market-favourite baselines. With full Kelly staking, the final bankroll is about $2.5\times$ the initial bankroll. However, this should be interpreted carefully. The gap between the full-Kelly and half-Kelly results comes from the same set of bets, so part of the stronger full-Kelly result is likely due to the higher variance of aggressive staking. The strategy is also very selective. It only bets on a small fraction of runners, which means the bets are spread across many race meetings. In practice, following this strategy would require betting consistently over a long period, so the return may not be as attractive once the time required is taken into account.

Taken together, the results suggest that the market is highly efficient, or at least that it already incorporates most of the publicly available information used by the model. That a small, out-of-sample edge survives at all is the more interesting finding: it is consistent with a market that prices public information well, leaving only limited and high-variance opportunities at the margins—which is the market inefficiency this project set out to probe, rather than a claim to reliably beat the market.

References

- [1] Hong Kong Jockey Club. *Official race results and betting information*. Available at: <https://racing.hkjc.com>.
- [2] Friedman, J. H. (2001). *Greedy function approximation: A gradient boosting machine*. The Annals of Statistics, 29(5), 1189–1232.
- [3] Shalev-Shwartz, S., and Ben-David, S. (2014). *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press.